

KRAFTON Jungle GameTech-Lab

WEEK 3

발표

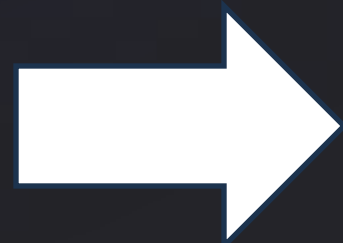
◆ Team 2 | 강명호 김연하 김형도 양현석

Week03

- 1 Texture Atlas
- 2 Index Buffer
- 3 Screen-Aligned Quad (Billboard)
- 4 FName System

Texture Atlas

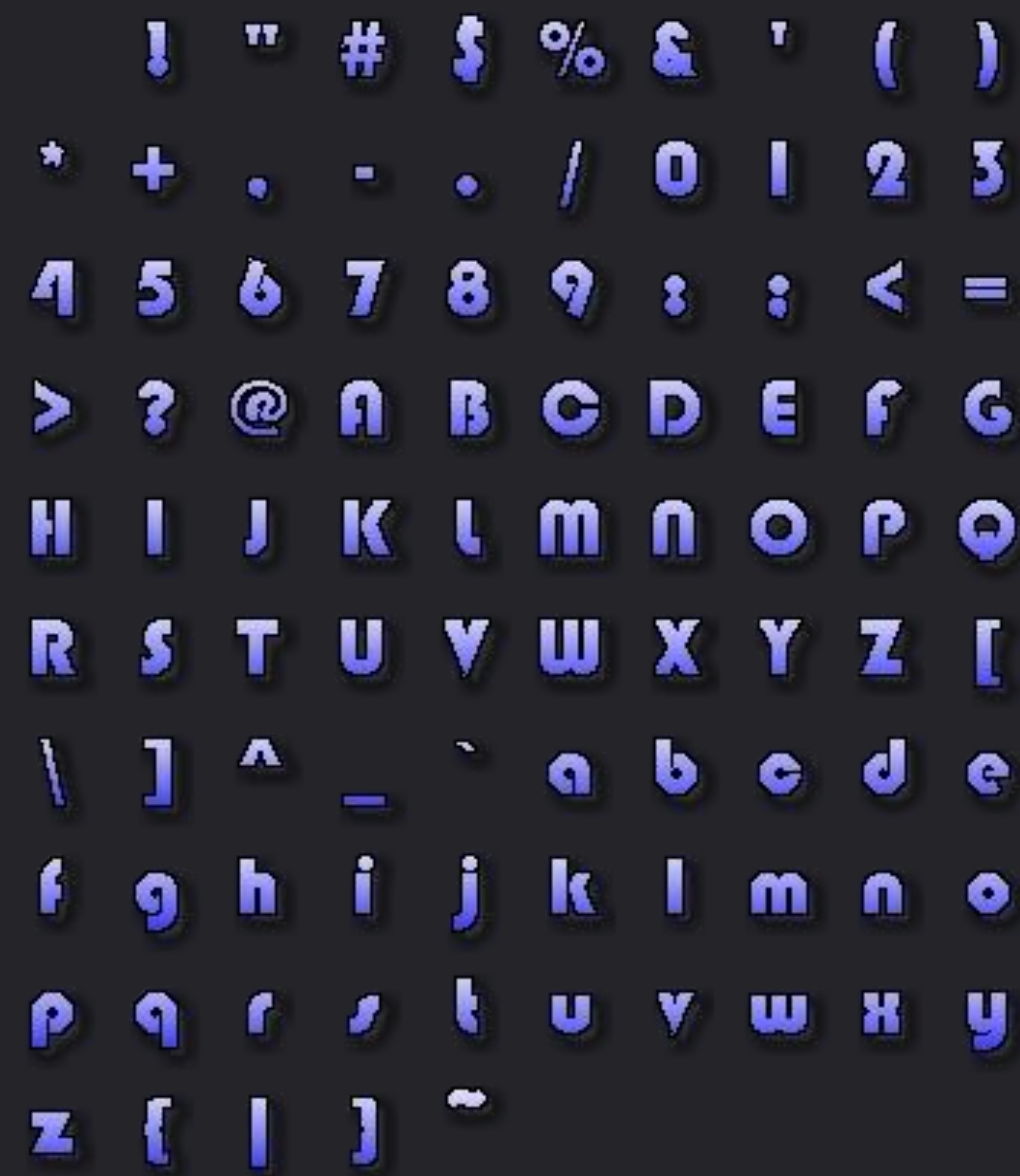
개별 Texture를 사용할 경우, Texture Binding 변경 및 Draw Call 증가
→ CPU의 렌더링 명령 제출 오버헤드가 커짐



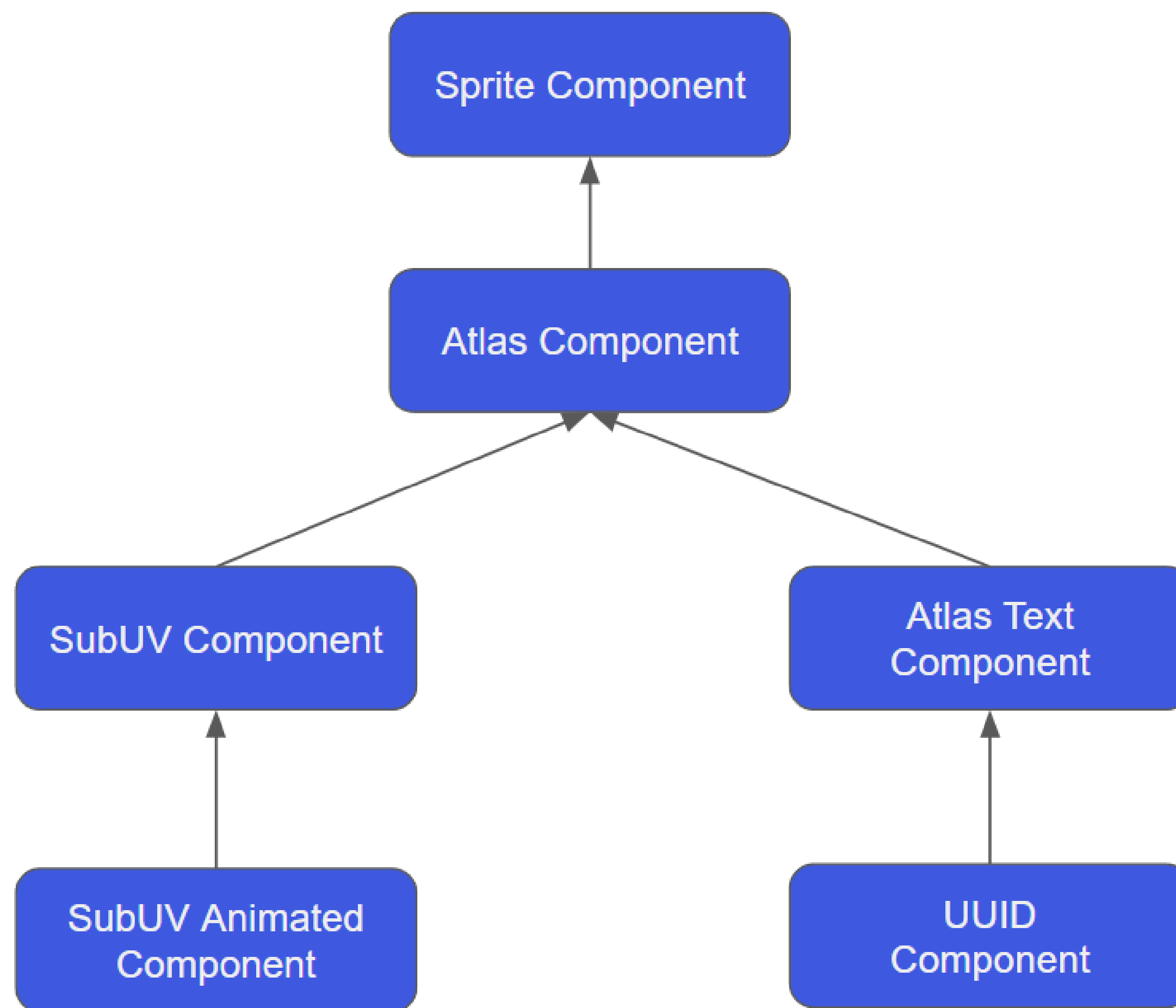
하나의 이미지에 여러 Texture를 집약해 놓는다

◆
Texture Atlas

다양한 목적으로 사용되는 Texture Atlas



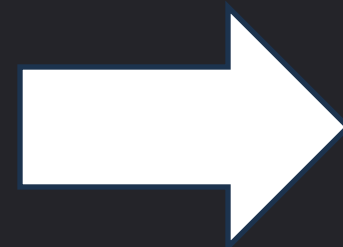
Texture Atlas



기존에는 입학시험때 주어졌던 Sphere.h를 사용했으나,

문제1. 기하 표현에 필요없는 데이터(색상값)이 들어있다

문제2. 정점 데이터가 중복되어 들어있다



인덱스 버퍼(Index Buffer) 도입

Topology를 구분해서 처리하기 위해 메쉬 파일에 metadata 추가

```
323, 343, 324, 344, 325, 345, 326, 346, 327, 347, 328, 348, 329, 349, 330, 350,  
331, 351, 332, 352, 333, 353, 334, 354, 335, 355, 336, 356, 337, 357, 338, 358,  
339, 359, 340, 360, 321, 341, 341, 341, 341, 361, 342, 362, 343, 363, 344, 364,  
345, 365, 346, 366, 347, 367, 348, 368, 349, 369, 350, 370, 351, 371, 352, 372,  
353, 373, 354, 374, 355, 375, 356, 376, 357, 377, 358, 378, 359, 379, 360, 380,  
341, 361, 361, 361, 361, 381, 362, 381, 363, 381, 364, 381, 365, 381, 366, 381,  
367, 381, 368, 381, 369, 381, 370, 381, 371, 381, 372, 381, 373, 381, 374, 381,  
375, 381, 376, 381, 377, 381, 378, 381, 379, 381, 380, 381, 361, 381  
};  
  
static constexpr EMeshPrimitiveTopology sphere_topology = EMeshPrimitiveTopology::TriangleStrip;  
static const uint32_t sphere_vertex_count = 382;  
static const uint32_t sphere_index_count = 878;
```

Index Buffer

인덱스 버퍼 도입 전: 약 201KB (Mesh 3종류)

 Sphere.h	2026-03-21 5:16 PM	H File	199 KB
 Triangle.h	2026-03-21 5:32 PM	H File	1 KB
 VertexSimple.h	2026-03-20 9:24 AM	H File	1 KB



인덱스 버퍼 도입 후: 약 60KB (Mesh 7종류)

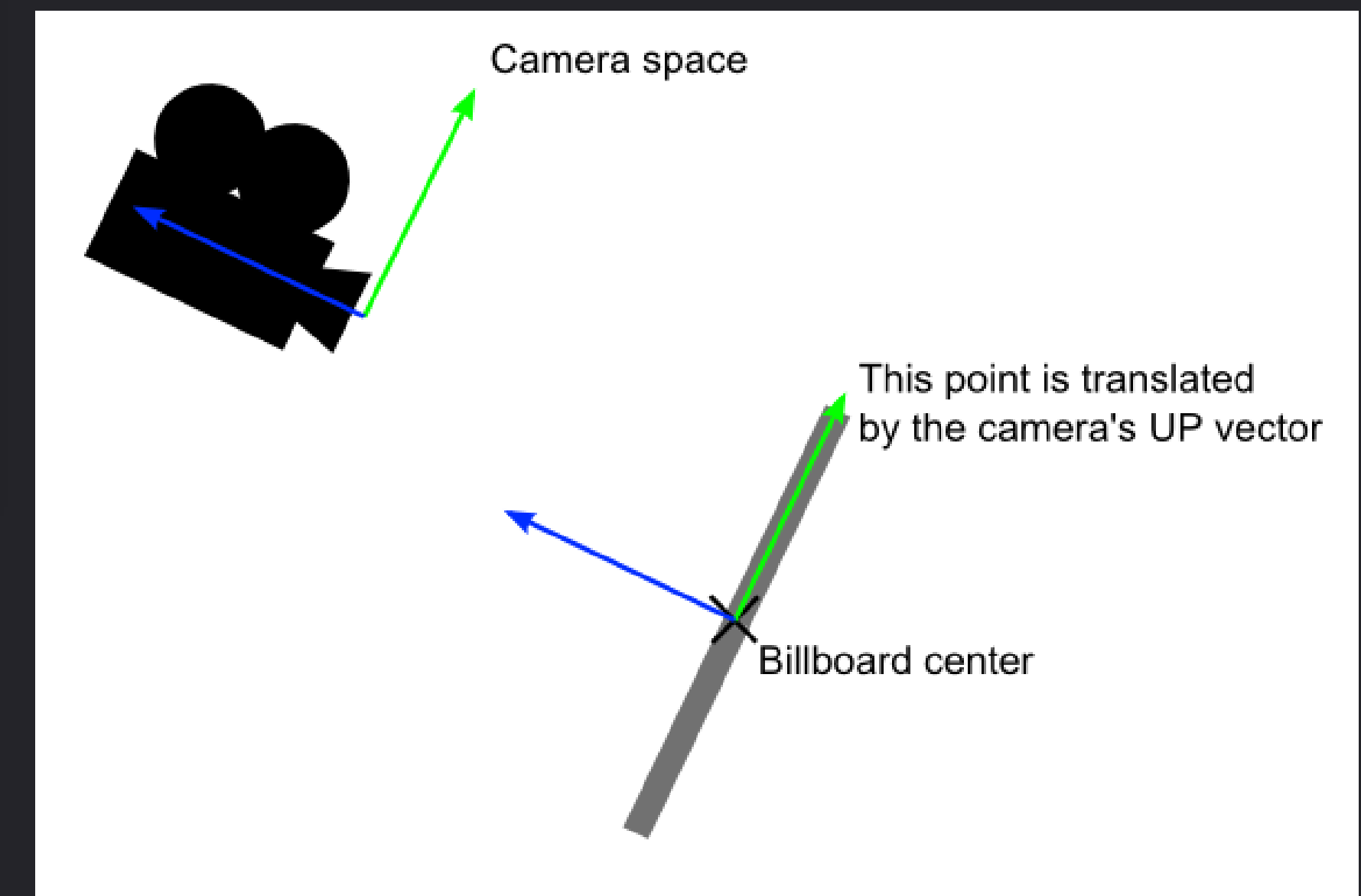
Name	Date modified	Type	Size
 Cone.h	2026-03-25 12:00 AM	H File	2 KB
 Cube.h	2026-03-24 10:12 PM	H File	1 KB
 Cylinder.h	2026-03-25 12:00 AM	H File	3 KB
 MeshPrimitiveTopology.h	2026-03-23 12:39 AM	H File	1 KB
 Quad.h	2026-03-24 10:12 PM	H File	1 KB
 Ring.h	2026-03-24 11:33 PM	H File	27 KB
 Sphere.h	2026-03-24 10:12 PM	H File	28 KB
 Triangle.h	2026-03-25 4:52 PM	H File	1 KB
 VertexSimple.h	2026-03-23 12:39 AM	H File	1 KB

방법1. 매 오브젝트마다 카메라를 직접 바라보도록 축을 계산

방법2. 카메라의 Right, Up 벡터를 사용



같은 프레임에서 카메라 기준 벡터를
공유할 수 있는 '방법2'를 택함



I FName

FName

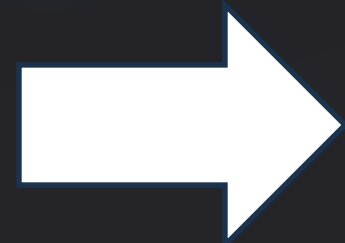
FName-Subsystem

Index	FString
0	None
1	Default
2	Actor
3	Spine_01
...	...
42	Weapon_socket
43	Boss
44	Mesh
45	



◆
프로그램 고도화

단순한 정적 Scene 렌더 프로그램이 아닌
Editor(편집) 프로그램이라는 관점에서 개발



Editor 프로그램으로서 사용성과 완성도를
높이는 방향으로 고도화

- 1 Input System : 다양한 편집 조작 지원 및 Context 기반 입력 처리
- 2 Batch Rendering : 렌더링 성능 및 효율 향상
- 3 Asset System : 에셋 중복 로드 방지 및 관리

상용 엔진 사용 경험을
기반으로
실용 · 직관적인
UX를 설계

Shortcuts	
현재 코드상 실제로 동작하는 에디터 단축키만 정리했습니다.	
Viewport Navigation	
Shortcut	Action
Mouse Right Drag	뷰포트 카메라 회전
Mouse Middle Drag	뷰포트 카메라 팬 이동
Alt + Mouse Left Drag	선택 대상을 기준으로 오빗 회전
Alt + Mouse Right Drag	카메라 돌리 인/아웃
Mouse Wheel	원근 카메라 FOV 또는 직교 카메라 높이 조절
Mouse Wheel while rotating	카메라 이동 속도 조절
W / A / S / D / Q / E	카메라 이동 (회전 중일 때만 적용)
F	현재 선택된 Actor 쪽으로 카메라 포커스
Selection	
Shortcut	Action
Mouse Left Click	Actor 단일 선택
Shift + Mouse Left Click	선택 추가
Ctrl + Mouse Left Click	선택 토글
Ctrl + Alt + Drag	박스 선택
Ctrl + Alt + Shift + Drag	기존 선택에 박스 선택 추가
Gizmo	
Shortcut	Action
Mouse Left Drag	기즈모 축 드래그 조작
Space	기즈모 타입 순환
X	월드/로컬 기즈모 모드 전환
Editor	
Shortcut	Action
Delete	선택된 Actor 삭제

참고: ImGui 입력창이 키보드를 잡고 있을 때는 일부 단축키가 동작하지 않습니다.

처리 순서



Interaction

사용자 입력

Windows API

윈도우 창 크기, 윈도우 종료 등 처리

Input System

남은 입력에 대하여 Raw Input을 Engine의 InputEvent로 변경

Input Router

InputEvent를 Queue에 저장 후 Context Layer들의 우선순위에 맞게 전송

Input Contexts

각 Context 별로 처리 가능한 Event를 Controller로 처리

Controllers

Context를 통해 실제 Action이 실행되는 계층

I Input System

◆
Input System

Context Layer
전송 순위



Global

삭제, 저장 등의 입력을 처리

Gizmo

Gizmo와 관련된 입력들을 처리

Navigation

카메라의 이동, 상태 변화 등에 대한 입력들을 처리

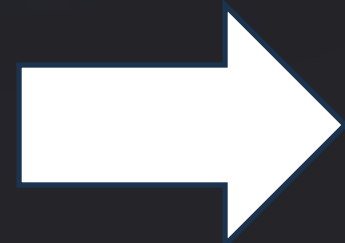
Selection

선택, 포커싱 등에 대한 입력들을 처리

"복수의 입력 상황을 우선순위 기반으로 안전하게 처리"

Batch Rendering

빈번한 Draw Call / State Change
→ CPU 오버헤드 증가



"렌더 요청이 들어올 때마다 즉시 Draw하지 않고,
같은 성격의 데이터를 모아 한 번에 렌더링"

Batch Rendering

Mesh Renderer

사전 정의된 7종의
기초 기하 Mesh를
타입별로 그룹화하여 처리

Line Renderer

Grid, Axes, AABB,
Wireframe 외곽선을
정렬 없이 단순 누적한 후
일괄적으로 Draw

Sprite Renderer

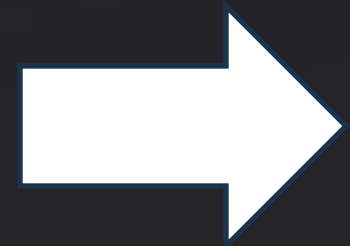
Depth를 기반으로
1차 정렬 후(Back-to-Front)
텍스처 리소스별로 2차 정렬

Text Renderer

Depth를 기반으로
1차 정렬 후(Back-to-Front)
텍스처 리소스별로 2차 정렬

Asset System

불필요한 리소스 Reload는
디스크 I/O 및 GPU 업로드 비용을 유발
→ 에디터 응답성 저하



"한번 Load한 파일을 불필요하게
다시 읽고, 다시 해석하지 않게 하자!"



리소스 로딩 경로 통합

에디터와 런타임에서 같은 리소스 로딩 경로를 재사용



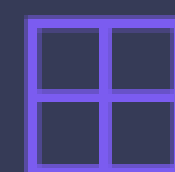
책임 분리

파일 경로, 디코드, GPU 리소스 생성,
캐싱 책임을 명확히 분리



컴포넌트 단순화

컴포넌트는 "무슨 리소스가 필요한가"만 알면 충분



에디터 연결

.Scene 저장/불러오기, Details 패널,
Content Browser 통합

- 1 FindLoader()로 타입별 로더 선택
- 2 SourceCache.GetOrLoad()로 파일 읽기 + 해시값 확보
- 3 FAssetKey 생성
- 4 기존 LoadedAssets 캐시 조회
- 5 없으면 로더 실행
- 6 생성된 UAsset을 캐시에 저장 후 반환

동일 폰트 경로를 참조하는
100개의 텍스트 컴포넌트가
하나의 Font Asset을 공유

UObject Classes					
Name	Live Count	Live Bytes	Peak Count	Peak Bytes	Avg Live Size
UAtlasTextComp	100	27.34 KB	100	27.34 KB	280.00 B
UCubeCompone	50	7.81 KB	50	7.81 KB	160.00 B
USphereComp	50	7.81 KB	50	7.81 KB	160.00 B
ACubeActor	50	3.52 KB	50	3.52 KB	72.00 B
ASphereActor	50	3.52 KB	50	3.52 KB	72.00 B
UFontAsset	1	368.00 B	1	368.00 B	368.00 B
UAssetManager	1	176.00 B	1	176.00 B	176.00 B

"동일한 리소스 경로를 참조하는 경우, 리소스는 한번만 생성되고 재사용 됨"

